

Práctica de Validaciones con Express Validator

Objetivo

Partiendo del proyecto "**Práctica de Relaciones con Sequelize**", se deberá incorporar la librería **express-validator** para implementar validaciones de entrada en los controladores ya desarrollados.

Criterios de Evaluación

1. Presentación del código

- El código debe estar limpio, ordenado y bien indentado.
- Uso obligatorio de try-catch en todos los controladores para un manejo adecuado de errores.
- Organización correcta del proyecto en carpetas:
 - **src/config** → configuración (conexión a la base de datos).
 - **src/models** → definición de modelos Sequelize.
 - **src/routes** → definición de rutas.
 - **src/controllers** → controladores.
 - **src/middlewares** → middlewares.
- Uso exclusivo de ESModules (import / export).
- El código debe ser funcional, modularizado y sin errores de ejecución.

2. Validaciones y lógica

- Validaciones correctamente implementadas con **express-validator**.
- Respuestas claras y códigos HTTP apropiados en cada operación:
 - **201 Created** → al crear un recurso.
 - **200 OK** → en consultas y actualizaciones exitosas.
 - **400 Bad Request** → errores de validación.
 - **404 Not Found** → recurso inexistente.
 - **500 Internal Server Error** → errores inesperados del servidor.
- Mensajes de éxito o error claros al realizar operaciones CRUD.

Entrega mediante Git y GitHub – Uso de ramas y commits

Teniendo en cuenta que al finalizar la práctica anterior las ramas main y develop quedaron sincronizadas, ahora se debe seguir el siguiente flujo:

- Crear una rama a partir de **develop** llamada **validaciones**.
- Realizar todo el desarrollo en la rama validaciones.
- Al finalizar el trabajo:
 - Hacer un merge limpio de validaciones hacia develop, sin conflictos.
 - Luego hacer un merge limpio de develop hacia main, para que ambas ramas queden sincronizadas nuevamente.
- Durante el desarrollo en la rama validaciones, se deben realizar **al menos 3 commits** con mensajes claros y descriptivos.

Consignas

1. Instalación de express-validator

Antes de comenzar con la implementación, se debe instalar la librería como dependencia del proyecto:

```
npm install express-validator
```

2. Middleware de manejo de errores de validación

Se debe crear un middleware reutilizable en **src/middlewares/** que centralice la verificación de errores producidos por express-validator. Este middleware debe ser utilizado en todas las rutas que tengan validaciones, evitando repetir la misma lógica dentro de cada controlador.

3. Validaciones en los controladores

Se deben incorporar validaciones de entrada en todos los controladores ya desarrollados. Las validaciones deben definirse en un array separado y aplicarse como middleware en la ruta correspondiente.

Para todas las rutas que reciban datos se deben aplicar, como mínimo, las siguientes validaciones:

- **IDs en parámetros (params):** validar que sean enteros positivos y que el recurso exista en la base de datos mediante una validación custom.
- **Campos obligatorios:** validar con `notEmpty()` y, cuando corresponda, con `isLength()` u otros métodos según el tipo de dato.
- **Campos con unicidad:** agregar una validación custom que consulte la base de datos y rechace el valor si ya existe en otro registro.
- **Validaciones personalizadas (custom):** agregar al menos una por modelo, según las reglas propias de cada entidad.